

Experiment 12 – Selenium Page Object Model (POM) with Behave

1. Module Name and Experiment Title

Module Name: Module 3 – Python BDD RESTful Automations

Experiment Number: Experiment 12

Experiment Title: Selenium Page Object Model (POM) with Behave

Programming Language: Python

Automation Tool: Selenium WebDriver

BDD Framework: Behave

Website: SauceDemo

Website URL: <https://www.saucedemo.com/>

2. Problem Statement

Develop a Selenium automation framework using the Page Object Model (POM) and Behave BDD framework to automate the login functionality of the SauceDemo website.

The implementation should separate page-specific elements and actions from test steps. Behave should execute the test scenario written in Gherkin, while Selenium WebDriver performs browser interactions.

The automation should:

- Open the SauceDemo website.
 - Enter valid login credentials.
 - Click the login button.
 - Verify successful redirection to the Products page.
 - Capture screenshots at important stages.
 - Close the browser after test execution.
-

3. Objective

The main objectives of this experiment are:

1. To implement the Page Object Model using Selenium and Python.
2. To integrate Selenium with the Behave BDD framework.
3. To write test scenarios using Gherkin syntax.
4. To separate page elements and actions from test steps.
5. To automate the login process on SauceDemo.
6. To verify successful login and capture screenshots.

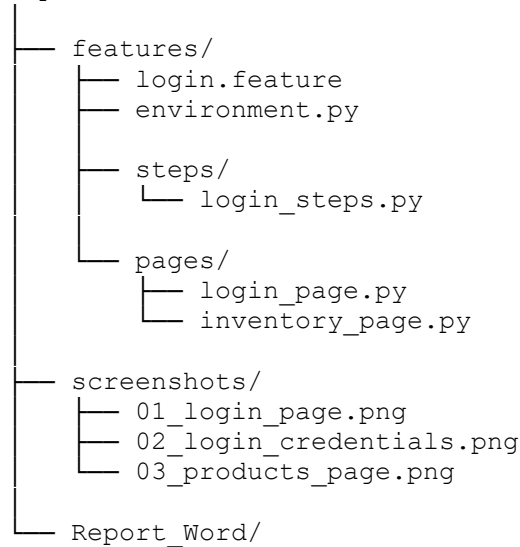
7. To execute the test scenario using the Behave command-line interface.

4. Tools, Software and Concepts Used

Tool / Concept	Description
Python	Programming language used to develop the automation framework.
Selenium WebDriver	Used to automate browser interactions.
Behave	BDD framework used to execute Gherkin scenarios.
Gherkin	Used to describe test scenarios in a readable format.
Page Object Model (POM)	Design pattern used to separate page elements and actions from test steps.
Google Chrome	Browser used for automation.
VS Code	Code editor used to develop and execute the project.
<code>time.sleep()</code>	Used to add short delays for demonstration and screenshot capture.
<code>assert</code>	Used to verify that the Products page is displayed.

Project Folder Structure

Experiment 12 - Selenium POM with Behave/



5. Source Code / Implementation Steps

Step 1: Create the Feature File

File: features/login.feature

The feature file describes the login scenario using Gherkin syntax.

Feature: SauceDemo Login using Page Object Model

Scenario: Successful login using valid credentials

Given the user opens the SauceDemo website

When the user enters valid login credentials

And the user clicks the login button

Then the user should be redirected to the products page

Step 2: Create the Login Page Class

File: features/pages/login_page.py

The LoginPage class contains the locators and methods required to interact with the login page.

```
from selenium.webdriver.common.by import By
```

```
class LoginPage:
```

```
    def __init__(self, driver):
        self.driver = driver

        self.username = (By.ID, "user-name")
        self.password = (By.ID, "password")
        self.login_button = (By.ID, "login-button")

    def open_website(self):
        self.driver.get("https://www.saucedemo.com/")

    def enter_username(self, username):
        self.driver.find_element(*self.username).send_keys(username)

    def enter_password(self, password):
        self.driver.find_element(*self.password).send_keys(password)

    def click_login(self):
        self.driver.find_element(*self.login_button).click()
```

Step 3: Create the Inventory Page Class

File: features/pages/inventory_page.py

The InventoryPage class contains methods to verify the Products page and retrieve its title.

```
from selenium.webdriver.common.by import By
```

```
class InventoryPage:
```

```
    def __init__(self, driver):
        self.driver = driver

        self.inventory_list = (
            By.CLASS_NAME,
            "inventory_list"
        )
```

```

def is_products_page_displayed(self):
    return (
        self.driver.find_element(*self.inventory_list).is_displayed()
        and "/inventory.html" in self.driver.current_url
    )

def get_page_title(self):
    return self.driver.title

```

Step 4: Configure Browser Setup and Teardown

File: features/environment.py

This file initializes the Chrome browser before each scenario and closes it after execution.

```

from selenium import webdriver
import os

def before_scenario(context, scenario):
    os.makedirs("screenshots", exist_ok=True)

    context.driver = webdriver.Chrome()
    context.driver.maximize_window()

def after_scenario(context, scenario):
    if hasattr(context, "driver"):
        context.driver.quit()
        print("Browser closed successfully.")

```

Step 5: Implement the Behave Step Definitions

File: features/steps/login_steps.py

This file connects the Gherkin steps to the page classes and performs the login test.

```

import time

from behave import given, when, then
from features.pages.login_page import LoginPage
from features.pages.inventory_page import InventoryPage

@given("the user opens the SauceDemo website")
def open_website(context):
    context.login_page = LoginPage(context.driver)
    context.login_page.open_website()

    time.sleep(3)
    context.driver.save_screenshot(
        "screenshots/01_login_page.png"
    )

    print("SauceDemo website opened successfully.")
    print("Screenshot 1: Login page saved.")

```

```

@when("the user enters valid login credentials")
def enter_credentials(context):
    context.login_page.enter_username("standard_user")
    time.sleep(3)

    context.login_page.enter_password("secret_sauce")
    time.sleep(3)

    context.driver.save_screenshot(
        "screenshots/02_login_credentials.png"
    )

    print("Login credentials entered successfully.")
    print("Screenshot 2: Login credentials saved.")

@when("the user clicks the login button")
def click_login(context):
    context.login_page.click_login()

    time.sleep(3)
    print("Login button clicked.")

@then("the user should be redirected to the products page")
def verify_products_page(context):
    context.inventory_page = InventoryPage(context.driver)

    assert context.inventory_page.is_products_page_displayed()

    time.sleep(2)
    context.driver.save_screenshot(
        "screenshots/03_products_page.png"
    )

    print("Products page verified successfully.")
    print("Screenshot 3: Products page saved.")
    print("Page title:", context.inventory_page.get_page_title())
    print("Page Object Model with Behave executed successfully!")

```

Step 6: Install the Required Libraries

Open the terminal in the experiment folder and execute:

```
pip install selenium behave
```

Step 7: Execute the Test Scenario

Run the following command from the experiment folder:

```
behave --no-capture
```

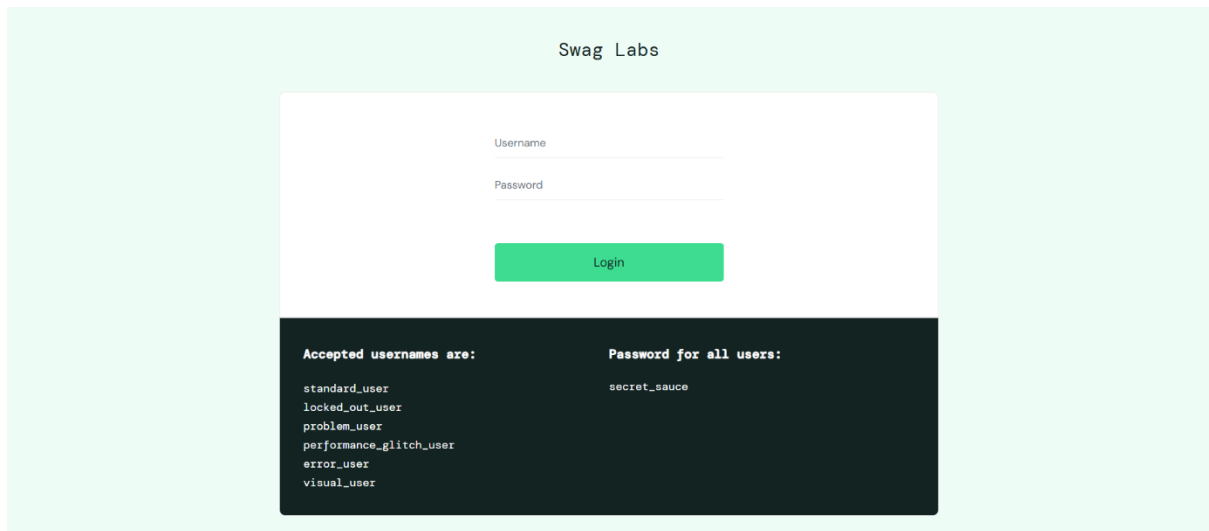
The `--no-capture` option displays the print statements from the step definitions in the terminal.

6. Output with Clearly Labelled Screenshots

The experiment was executed successfully using Behave and Selenium WebDriver. The following screenshots document the execution.

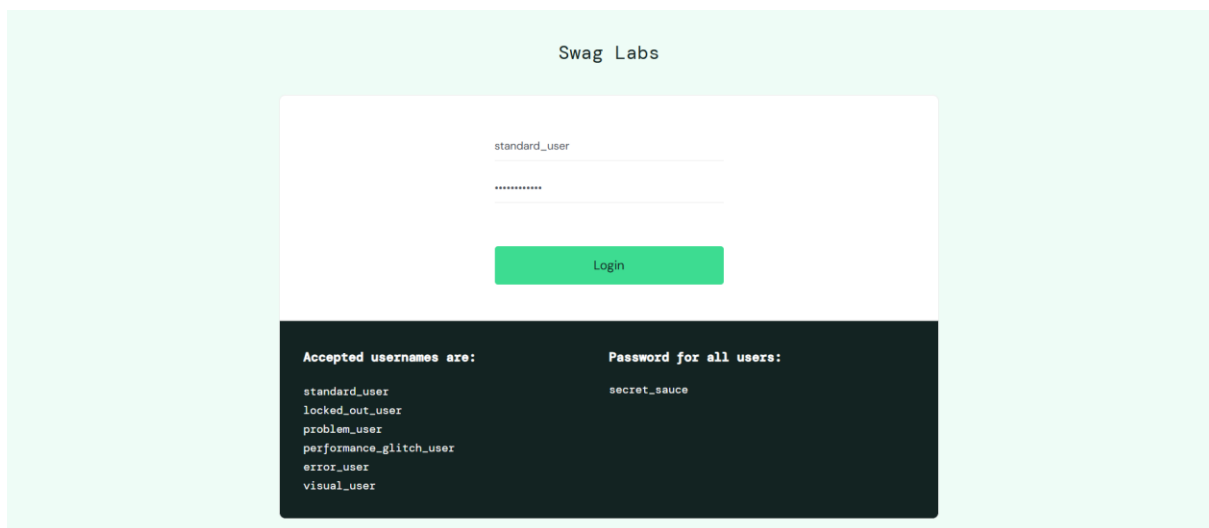
Screenshot 1: SauceDemo Login Page

Description: The SauceDemo login page is opened in Chrome. The browser is maximized before the test begins.



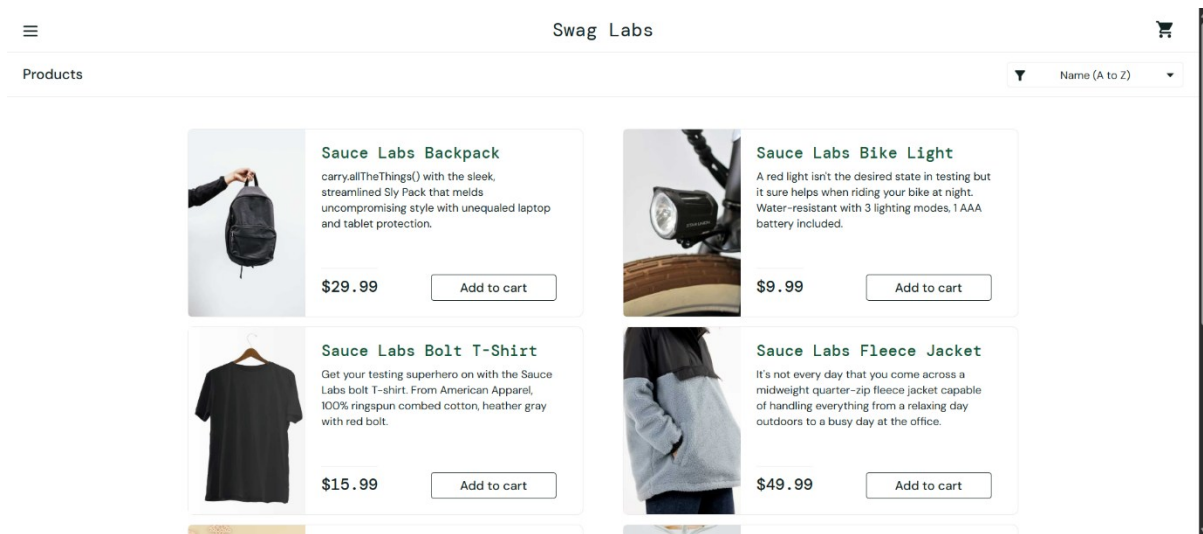
Screenshot 2: Login Credentials Entered

Description: The valid username `standard_user` and password `secret_sauce` are entered into the login form before clicking the Login button.



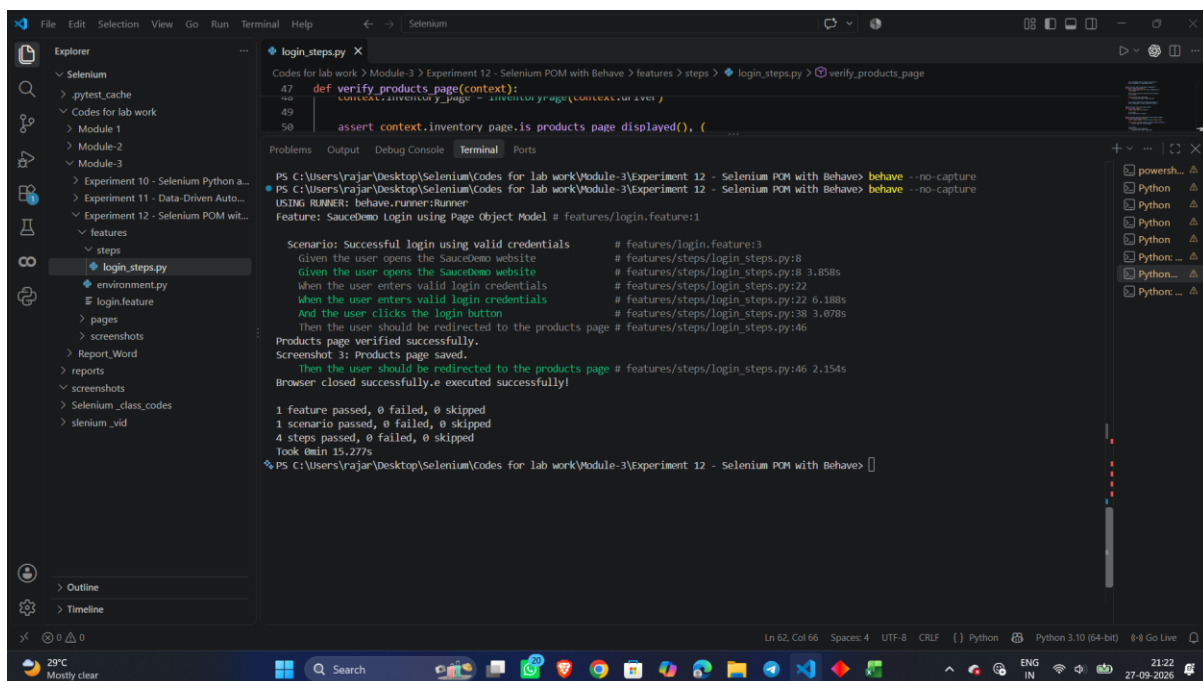
Screenshot 3: Products Page After Successful Login

Description: After submitting the valid credentials, the application redirects to the Products page. The test verifies that the inventory page is displayed.



Screenshot 4: Behave Terminal Execution Output

Description: The terminal displays the execution results. The feature, scenario, and all four steps passed successfully. The browser is closed after execution.



Terminal Output:

```

USING RUNNER: behave.runner:Runner

```

Feature: SauceDemo Login using Page Object Model

Scenario: Successful login using valid credentials
Given the user opens the SauceDemo website

```
When the user enters valid login credentials
And the user clicks the login button
Then the user should be redirected to the products page
```

```
SauceDemo website opened successfully.
Screenshot 1: Login page saved.
Login credentials entered successfully.
Screenshot 2: Login credentials saved.
Login button clicked.
Products page verified successfully.
Screenshot 3: Products page saved.
Page title: Swag Labs
Page Object Model with Behave executed successfully!
Browser closed successfully.
```

```
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped
Took 15.277s
```

Note: The terminal output above is formatted from the successful execution shown in your screenshot. Keep the actual terminal screenshot in the report as the evidence of execution.

7. Result, Observation and Conclusion

Result

The Selenium Page Object Model framework was successfully implemented and integrated with the Behave BDD framework. The login scenario was executed on the SauceDemo website using valid credentials.

The test successfully opened the website, entered the login credentials, clicked the Login button, and verified that the Products page was displayed.

The execution completed with **1 feature passed, 1 scenario passed, and 4 steps passed**, with no failures or skipped steps.

Observation

1. The Gherkin feature file made the test scenario readable and easy to understand.
2. The Page Object Model separated page-specific actions from the test steps.
3. The `LoginPage` class handled login-related operations.
4. The `InventoryPage` class handled Products page verification.
5. Behave successfully executed the scenario using the defined step functions.
6. Screenshots were captured at the login page, credential entry, and Products page stages.
7. The browser was closed automatically after the scenario completed.

Conclusion

The experiment demonstrated the implementation of Selenium automation using the Page Object Model and Behave BDD framework. The login functionality of the SauceDemo website was automated and verified successfully.

The experiment provided practical experience in organizing Selenium automation code, writing Gherkin scenarios, implementing page classes, and executing BDD tests using Behave.